3-2 개발 방법 안내서

book-trailer(NOMAD Book Trailer) 개발 방법

develop/book-trailer/book-trailer_develop.md

작성일 2026-10-15. 이 문서는 `D:\WorkSpace\platform\docs\products\book-trailer\`의 기획을 멀티에이전트 베이스(`D:\WorkSpace\DEV\MultiAgent-Service-Fullstack-Tree-Framwork`)로 옮겨 book-trailer 개발을 시작하는 방법을 처음부터 끝까지 안내합니다. 형식은 `develop/nomad-docs/nomad-docs_develop.md`와 같고, 이 제품에만 있는 부분(렌더러, 영상 디자인 토큰, 권리·인용 규칙)을 더했습니다. 근거가 없는 값은 "제안", 확인하지 못한 값은 "미확인"으로 표시합니다.

0. 한눈에 보기

시작은 setup 스크립트 1개 실행입니다. 둘 중 하나만 실행하면 됩니다.

환경	명령
Windows PowerShell	`pwsh -File D:\WorkSpace\platform\docs\develop\book-trailer\setup-book-trailer.ps1`
Git Bash / Linux	`bash /d/WorkSpace/platform/docs/develop/book-trailer/setup-book-trailer.sh`

먼저 `-DryRun`(PowerShell) 또는 `--dry-run`(Git Bash)으로 실행해 할 일을 확인한 뒤 실제로 실행하시길 권합니다.

```
D:\WorkSpace\platform\docs\develop\book-trailer\       platform에 남는 안내서, 도구, 재료
 ├─── book-trailer_develop.md      이 문서(개발 폴더로 옮기지 않음)
 ├─── setup-book-trailer.sh        설치 스크립트(Git Bash / Linux)
 ├─── setup-book-trailer.ps1       설치 스크립트(Windows PowerShell)
 └─── book-trailer-root\           이 폴더 "안의 내용물"이 개발 폴더 루트에 들어갑니다
     ├─── docs\  supabase\  infra\  .claude\
                 │
                 │ setup 스크립트가 복사(안의 내용물)
                 ▼
D:\WorkSpace\DEV\book-trailer\                          베이스를 복사해 만든 개발 폴더
 ├─── docs\  supabase\  infra\  .claude\skills\llm-wiki\  ← book-trailer-root에서 들어온 것
 └─── apps\  _shared\  _templates\  .claude\agents\  init-project.sh ...  ← 베이스에 원래 있던 것
```

단계	내용	절
0.5	개발 전 린 검증(대행 판매, 수동 제작)	1.1
준비	도구 설치, API 키, Remotion 라이선스 확인	2
1~2	setup 스크립트 실행(베이스 복사, 초기화, 묶음 붙여넣기, docs/etc 삭제, 에이전트 모델 지정)	3, 4
3	최종 트리 확인, 기획 경로 → 개발 경로	5
4	project-profile 작성	6
5	로컬 DB 기동, SQL 적용, RLS 스모크	7
6	착수 전 결정 확정, 산출물 승인	8
7	작업(60개)을 task로 변환	9
8	웨이브별 개발 사이클	10
-	AI 계층, 렌더 파이프라인, 개발 위키, 디자인(앱 + 영상), 원본 이식, 공용 모듈, 검증, 위험	11~17
-	체크리스트	18

단계	내용	절
-	인증 단계: Mock → Google → 카카오·네이버	19

1. 개요

항목	내용
제품	NOMAD Book Trailer(가칭, D-1). 원고(PDF·EPUB·DOCX·MD·TXT)나 FLORA 책을 넣으면 원고 분석 → 훅 브리프 3안 → 장르 문법 스토리보드 → 장면 이미지·내레이션·자막·음악 → 쇼츠(9:16)·유튜브(16:9)·인스타(1:1, 4:5)·표지 루프 렌더 → 구매 링크 클릭 측정까지 하는 북트레일러 자동화 SaaS. 공개 베타, 가입 개방
통합 원본	없음(신규 제품). 이식 원천: nomad-video(미디어 파이프라인 설계), toon-factory(장면 단위 저장·서버 렌더 워터마크·라이선스 CHECK), flora-publishing(업로드·그래프 패턴, FLORA 연동), nomad-docs(결정성 원칙), 리뷰 2건(NOMAD Shorts Maker, Toon-Factory)
모듈러 앱 id / 스키마	`trailer` / `trailer` (번호 2600, SQLSTATE `TR`)
이번 개발 방식	단독 Supabase 프로젝트 1개(모놀리스, `public` 스키마)로 시작하는 것을 제안합니다. 기획의 1차 출시안은 모듈러이고 (`README.md` 5장), 모놀리스는 "베타를 독립 Supabase 프로젝트로 먼저 띄울 때" 쓰는 경로입니다. 함수 이름·시그니처는 같고 `SUPABASE_DB_SCHEMA=public` 만 다릅니다. 이 경로에서는 FLORA 가져오기(TR-06-056)가 `TR010` 을 냅니다
개발 근거	`docs/file/` 의 산출물 27개(등록부 `docs/file/_index.md`). 정본은 이 산출물입니다
런타임 구성	web(Next.js, Vercel), 오케스트레이터(Node 20, `core.jobs` 클레임, 그래프·이미지·TTS 호출), 렌더러(Node 20 + Remotion + Chrome Headless Shell, Docker), AI 서비스(FastAPI + LangGraph, 공용 설치), 공용 Dify·음성·오픈소스 LLM 서버
AI 범위	원고 분석 `trailer.digest` (장 요약·인물·주제 + RAG 색인 + 책 설정 위키 초안), 브리프 3안 `trailer.brief` (인용 원고 대조, 스포일러 수준, compliance), 스토리보드 `trailer.storyboard` (문법 6종, 글자 노출 시간 공식), 메타데이터 `trailer.meta` (플랫폼 4개), 장면 이미지(fal Flux, 스타일 고정), 내레이션(같은 목소리), 책 근거 코치 채팅(Dify `trailer-chat`, 출처 장·문단), 음성 대화·받아쓰기
AI 금지 범위	원고에 없는 문장을 인용으로 표시, 근거 없는 판매·순위·수상 문구, 정가 15% 초과 할인 표현, 실존 인물·특정 작가 화풍 이미지, AI 표시 없는 렌더(`docs/file/04-구현/CLAUDE.md` 2절)

이 문서는 "어떻게 시작하고 진행하는가"를 다룹니다. "무엇을 만드는가"는 묶음 안의 산출물(`docs/file/`)이 정본입니다.

이름 주의: `trailer` 는 앱 id(가입 `signup_app='trailer'`, 큐 `trailer.ai` · `trailer.render` · `trailer.media`, 멱등 키 `run:trailer:*`)입니다. 제품 폴더 이름은 `book-trailer` 입니다.

1.1 개발 전 린 검증 (제안)

개발 작업 60개는 큰 투자입니다. 아래 검증을 개발과 병행하거나 먼저 하면 위험이 줄어듭니다(`Price_Plans/book-trailer/PRICE_PLAN.md` 5장·10장 TR-P-1).

순서	검증	방법	통과 기준(제안)
1	수요	크몽 "출판업 영상" 카테고리에 "AI 북트레일러 30초 99,000원" 상품 등록, 1인 출판사 커뮤니티 공지	4주 안 문의 10건, 주문 3건
2	품질	주문 원고로 수동 제작: Claude로 브리프·스토리보드, 이미지 도구, TTS, 기존 편집기로 합성	고객 수정 요청 1회 이내, 재주문 의향
3	차별점	같은 원고로 범용 AI 숏폼 도구 결과와 비교(인용 정확도, 스포일러, 규격 4종)	출판사 3곳이 "이쪽이 낫다"
4	가격	대행 S-30/S-60과 구독 creator 29,000원 안내, 사전 예약	사전 예약 20곳
5	정부지원	위 결과(매출·사전 예약)를 근거로 창업·콘텐츠 지원사업 신청서 작성	신청서 근거 자료 확보

실행 문서(크몽 상품 원고, 수동 제작 절차와 프롬프트, 주문 요청서·인터뷰, 판정 기준)는 `Price_Plans/book-trailer/01_VALIDATION_PLAN.md` ~ `04_INTAKE_AND_INTERVIEW.md`에 있습니다. 수동 제작에서 반복되는 단계가 그대로 이 제품의 그래프·잡 순서입니다. 손으로 한 결과(좋은 브리프, 잘 된 장면 구성)를 평가셋(TR-06-022·023·025 수용 기준)에 넣으면 개발이 빨라집니다.

2. 사전 준비

도구	용도	확인(Git Bash)	확인(PowerShell)
Git for Windows(Git Bash)	베이스의 `.sh` 스크립트 실행	`bash --version`	`& "$env:ProgramFiles\Git\bin\bash.exe" --version`
git, jq	저장소, 베이스 게이트·무결성 점검	`git --version; jq --version`	같음
Python 3	베이스 self-test, setup의 모델 지정 단계	`python --version`	같음
Node.js 20 LTS + pnpm(제안)	web, 오케스트레이터, 렌더러, 컴포지션 패키지, SQL 체커	`node -v; pnpm -v`	같음
Python 3.12 + uv(제안)	`apps/api` (FastAPI, LangGraph)	`uv --version`	같음
Supabase CLI	로컬 DB	`supabase --version`	같음
Docker Desktop	Supabase 로컬, 렌더러 컨테이너, Dify, 오픈소스 LLM·임베딩·음성 서버	`docker version`	같음
NVIDIA GPU + Container Toolkit(WSL2)	오픈소스 LLM·음성 서버를 직접 띄울 때만	`nvidia-smi`	같음

API 키(서버 환경변수로만, `docs/file/03-설계/02_ARCHITECTURE.md` 8절) - Anthropic, OpenAI, Google: `apps/api` 전용 - fal: 오케스트레이터 전용(이미지) - ElevenLabs: web 서버(음성 대화)와 오케스트레이터(내레이션) - Toss Payments, Supabase, 서명 토큰 키쌍 `AI_TOKEN_PRIVATE_KEY`(Ed25519, web 서버 전용) - `IP_HASH_SALT_SEED`(구매 링크 클릭 IP 해시용)

Remotion 라이선스: 개인·직원 3명 이하 영리 조직은 무료, 그 밖은 Company License입니다(D-2, `https://github.com/remotion-dev/remotion/blob/main/LICENSE.md`). 직원이 4명이 되는 시점에 구매하고 원가표(`Price_Plans/book-trailer/PRICE_PLAN.md` 3장)를 갱신합니다. 개발을 맡긴 외주 인력도 인원에 들어가는지는 라이선스 원문에서 확인합니다(미확인).

공용 인프라 문서

인프라	문서
Dify(공용 1개)	`D:\WorkSpace\platform\docs\develop\_shared\dify\README.md`
오픈소스 음성 서버	`...\_shared\voice\README.md`
오픈소스 LLM 서버·임베딩·상용 키	`...\_shared\llm\README.md`
로그인 공용 지시서	`...\_shared\auth\README.md` (19장)

3. 1단계: 베이스 복사와 초기화 (setup 스크립트가 처리)

순서	동작
1	베이스 원본을 `D:\WorkSpace\DEV\book-trailer`로 복사(대상이 있으면 중단)

순서	동작
2	**복사본 안의** `init-project.sh book-trailer --reset-git` 실행
3	`book-trailer-root` 안의 내용물(`docs` , `supabase` , `infra` , `.claude`)을 복사본 루트에 붙여넣기
4	복사본의 `docs/etc` (베이스 설명서 PDF) 삭제
5	에이전트 모델 지정: `plan-challenger` → `claude-fable-5-1` (고문), `nextjs-expert` · `fastapi-expert` · `supabase-expert` → `sonnet` (부)
6	무결성 점검(`_shared/check-invariants.sh`)

setup 스크립트는 커밋, 원격 푸시, 의존성 설치, 원격 DB 변경을 하지 않습니다. **반드시 복사본 안의 `init-project.sh` 를 실행합니다.** 베이스 원본에서 실행하면 원본이 초기화되고 `--reset-git` 이면 git 이력이 지워집니다. 고문을 Opus로 쓰려면 setup 뒤 `.claude/agents/plan-challenger.md` 를 `model: opus` 로 되돌립니다.

4. 2단계: 묶음 붙여넣기 (setup 스크립트가 처리)

- `book-trailer-root` 폴더 자체가 아니라 **그 안의 내용물**이 개발 폴더 루트에 들어갑니다. `docs` 와 `supabase` 는 베이스의 같은 폴더와 합쳐집니다.
- 덮어쓰는 파일은 `docs/file/_index.md` 하나뿐입니다(init이 만든 빈 등록부를 완성본으로 교체).
- 손으로 하는 방법은 부록 A에 있습니다.

5. 3단계: 이관 경로와 최종 트리

5.1 이관 경로 (원본 기준 `D:\WorkSpace\platform\`)

원본	개발 폴더 안 경로	ID	산출물명
`docs/products/book-trailer/README.md`	`docs/file/01-관리/README.md`	DOC-MGT-001	제품 개요
`docs/products/book-trailer/06_AGENT_PLAN.md`	`docs/file/01-관리/06_AGENT_PLAN.md`	DOC-MGT-002	작업 계획(WBS)
`docs/products/book-trailer/01_PRD.md`	`docs/file/02-요구사항/01_PRD.md`	DOC-REQ-001	요구사항정의서
`docs/products/book-trailer/02_ARCHITECTURE.md`	`docs/file/03-설계/02_ARCHITECTURE.md`	DOC-DES-001	아키텍처설계서
`docs/products/book-trailer/03_SCREENS_UX.md`	`docs/file/03-설계/03_SCREENS_UX.md`	DOC-DES-002	화면설계서
`docs/products/book-trailer/04_API.md`	`docs/file/03-설계/04_API.md`	DOC-DES-003	인터페이스설계서
`docs/products/book-trailer/05_DATA_MODEL.md`	`docs/file/03-설계/05_DATA_MODEL.md`	DOC-DES-004	데이터설계서
`docs/products/book-trailer/08_DESIGN_SYSTEM.md`	`docs/file/03-설계/08_DESIGN_SYSTEM.md`	DOC-DES-005	디자인시스템설계서(앱 토큰 + 영상 토큰)
`docs/products/_platform-core/` 9개	`docs/file/03-설계/공용코어/`	DOC-CORE-001~009	공용 코어 명세(009 `AUTH_ROADMAP.md`)
`docs/products/_design-guide/` 5개 + `DASHBOARD_PATTERNS.md`	`docs/file/03-설계/디자인/`	DOC-DSN-001~005, 008	디자인 기준

원본	개발 폴더 안 경로	ID	산출물명
(생성) DESIGN.md, COMPONENTS.md	`docs/file/03-설계/디자인/`	DOC-DSN-006~007	앱 디자인 토큰·컴포넌트 초기값 (`08_DESIGN_SYSTEM.md` 3.7절·11.3절에서 복사)
`docs/products/book-trailer/CLAUDE.md`	`docs/file/04-구현/CLAUDE.md`	DOC-IMP-001	개발지침
`docs/products/book-trailer/07_SOURCE_MIGRATION.md`	`docs/file/04-구현/07_SOURCE_MIGRATION.md`	DOC-IMP-002	원본 이식 계획
`docs/products/00_MERGE_ANALYSIS.md`	`docs/reference/00_MERGE_ANALYSIS.md`	참고	통합 분석, 확정 결정
`docs/reviews/NOMAD_Shorts_Maker.md` , `Toon-Factory.md`	`docs/reference/reviews/`	참고	이식 원천 리뷰 2건
`.../book-trailer/supabase/monolith/migrations/0001_core_lite.sql`	`supabase/migrations/20261013000001_core_lite.sql`	-	DB(코어 축소판, SECTION 9 affiliate만 제거)
`.../0002_trailer_domain.sql`	`supabase/migrations/20261013000002_trailer_domain.sql`	-	DB(도메인 17테이블, 카탈로그 seed: 문법 6, 템플릿 6, runtime_config 8)
`.../monolith/seed.sql`	`supabase/seed.sql`	-	apps 1행, 플랜 free/creator/publisher, 크레딧팩 3종, AI 라우팅 16행
`.../monolith/tests/rls_smoke.sql`	`supabase/tests/rls_smoke.sql`	-	RLS·권한 회귀 124항목
`docs/develop/_shared/llm-wiki/`	`docs/wiki/WIKI_SCHEMA.md` , `.claude/skills/llm-wiki/SKILL.md`	-	개발 위키
(생성) 위키 초기 페이지, Dify 앱 설명서	`docs/wiki/` , `infra/dify/README.md`	-	산출물 요약 27개, 채팅 앱 설정

- 묶음은 `D:\WorkSpace\platform\docs\develop\_shared\tools\build_root.py book-trailer` 로 만들었습니다(2026-10-15, 산출물 27개, SQL 4개). 같은 날 이 스크립트에 book-trailer 등록 행(`REVIEWS` , `APP_ID`)과 선택 기획서 `08_DESIGN_SYSTEM.md` (DOC-DES-005), `###` 수준 "초기값" 절 읽기를 추가했습니다. 다른 제품의 묶음 결과는 바뀌지 않습니다(nomad-docs·local-news 초기값 동일 확인).
- 경로 검사(`build_root.py --check book-trailer`): 문서 경로 399개, 묶음 안 누락 0, 절대경로 누락 0.
- 복사하지 않은 파일(예: `_supabase-modular/migrations/2600_trailer.sql` , nomad-video·toon-factory 기획서)을 가리키는 참조는 `D:/WorkSpace/platform/...` 절대경로로 바뀌어 계속 열 수 있습니다.
- RLS 러너 `run-smoke.mjs` 는 묶음에 들어가지 않습니다(platform 저장소 경로를 전제). 개발 폴더 러너는 TR-06-002가 만들고, 그 전에는 7.3절 방법을 씁니다.

5.2 최종 트리 (setup 실행 후 `D:\WorkSpace\DEV\book-trailer\`)

```
D:\WorkSpace\DEV\book-trailer\
├── CLAUDE.md  init-project.sh  INSTALL.md  README.md      [베이스]
├── .claude\agents\  commands\  skills\                    [베이스] + skills\llm-wiki\ [묶음]
├── _shared\  _templates\                                  [베이스]
├── apps\
│   ├── web\                          [베이스] Next.js(W0에서 스캐폴딩)
│   └── api\                          [베이스] FastAPI(W0에서 스캐폴딩)
│                                  (오케스트레이터·렌더러·컴포지션은 아래 표 경로로 추가)
├── docs\
│   ├── reference\00_MERGE_ANALYSIS.md, reviews\(2)        [묶음]
```

```
│   ├── file\_index.md                [묶음] 산출물 등록부(27개)
│   │   ├── 01-관리\README.md, 06_AGENT_PLAN.md
│   │   ├── 02-요구사항\01_PRD.md
│   │   ├── 03-설계\02~05, 08_DESIGN_SYSTEM.md, 공용코어\(9), 디자인\(6 + DESIGN.md, COMPONENTS.md)
│   │   └── 04-구현\CLAUDE.md, 07_SOURCE_MIGRATION.md
│   └── wiki\                        [묶음] WIKI_SCHEMA, index, log, sources\(27)
├── infra\dify\README.md             [묶음] DSL은 TR-06-040이 trailer-chat.yml로 추가
├── supabase\migrations\(2), seed.sql, tests\rls_smoke.sql   [묶음]
└── tasks\  _local\                  [베이스]
```

기획서 경로를 개발 폴더에서는 이렇게 읽습니다. task의 `write_scope`에도 오른쪽 값을 씁니다.

기획서 경로(06장 약어)	개발 폴더 경로
`web/` = `apps/web/`	`apps/web/`
`api/` = `apps/api/`	`apps/api/`
`orch/` = `workers/trailer-orchestrator/`	`workers/trailer-orchestrator/`
`rend/` = `services/trailer-renderer/`	`services/trailer-renderer/`
`comp/` = `packages/trailer-compositions/`	`packages/trailer-compositions/`
`packages/trailer-rules/`	같음
`supabase/monolith/migrations/0001`, `0002`	`supabase/migrations/20261013000001_*`, `20261013000002_*`

모노레포 도구는 pnpm workspace(제안)입니다. `pnpm-workspace.yaml`에 `apps/*`, `workers/*`, `services/*`, `packages/*`를 넣고, Remotion 패키지는 한 버전으로 고정합니다(`CLAUDE.md` 5절).

6. 4단계: project-profile 작성

`_shared/project-profile.md`에 다음 값을 채웁니다.

항목	값	근거
적용 표준	platform 통합 기획 규칙(DOC_TEMPLATE)	`docs/file/_index.md`
웹	Next.js 16.2.x, React 19.2, TypeScript strict, Tailwind CSS v4(4.x 고정), shadcn/ui(Tailwind CSS v4 지원 버전), zod 3.x, lucide-react, `@supabase/ssr`, `@remotion/player`(미리보기만)	`02_ARCHITECTURE.md` 1절, `CLAUDE.md` 5절
3D	three.js + React Three Fiber(React 19 호환), `hero3d/` 안에서만	같은 곳
오케스트레이터	Node 20 LTS + TS(`workers/trailer-orchestrator`, 큐 `trailer.ai`)	`02_ARCHITECTURE.md` 4절
렌더러	Node 20 + Remotion 4.0.x + Chrome Headless Shell, Docker 이미지에 OFL 서체 8종 포함(큐 `trailer.render`, `trailer.media`)	`02_ARCHITECTURE.md` 1·4절, `08_DESIGN_SYSTEM.md` 4.2절
AI 서비스	FastAPI + Python 3.12 + LangChain/LangGraph, 원고 파서(pypdf 계열, ebooklib, python-docx)	`02_ARCHITECTURE.md` 1절
DB	Supabase(PostgreSQL + pgvector), 단독 프로젝트, `public` 스키마, `SUPABASE_DB_SCHEMA=public`	`05_DATA_MODEL.md` 8절
명령	`pnpm -F web dev / build / lint / typecheck / test`, `pnpm -F trailer-renderer bench`, api: `uv sync`, `uvicorn app.main:app --reload`, `pytest apps/api -k trailer`, `ruff check apps/api`, DB: `supabase start`, `supabase db reset`, `supabase gen types typescript --local --schema public > apps/web/src/types/supabase.ts`	`CLAUDE.md` 6절 (pnpm·uv는 제안)

항목	값	근거
모델 핀	claude-main `claude-opus-5-5`, claude-reviewer `claude-sonnet-5-5` (변경하지 않음)	베이스 `_shared/backends.json`

버전은 `latest`로 두지 않고 lockfile로 고정합니다. **Tailwind는 반드시 Tailwind CSS v4**입니다. `tailwind.config.js`와 v3 문법을 쓰지 않습니다.

7. 5단계: 로컬 DB

7.1 기동과 적용

순서	명령	설명
1	`supabase init`	`config.toml`이 없을 때만. 기존 `migrations`, `seed.sql`, `tests`는 그대로
2	`supabase start`	Docker로 로컬 Supabase 기동
3	`supabase db reset`	`migrations/`를 이름순으로 적용하고 `seed.sql`을 넣습니다
4	`supabase gen types typescript --local --schema public > apps/web/src/types/supabase.ts`	타입 생성

- `core_lite`는 SECTION 9(affiliate)만 뺐습니다. 크레딧, 잡, Storage(`org-files`, `exports`), OAuth state, 자격증명, AI(pgvector), 음성을 모두 유지합니다.
- `0002` 말미 카탈로그 seed: 문법 6종(`grammars`), 템플릿 6종(`templates`), `runtime_config` 8행(동시 실행 4, 동시 렌더 2, 인용 80/300자, 원고 600,000자, 승인 링크 7일, 렌더 보존 14/90일). 음악 카탈로그는 비어 있습니다(D-7 확보 후 운영자 콘솔 TR-06-045로 등록).
- `seed.sql`은 라우팅 행이 16개가 아니거나 오픈소스 강제 행의 체인에 상용 제공자가 있으면 스스로 예외를 냅니다.
- 제품 버킷 3개는 `0002` 9절이 만듭니다: `trailer-media`(비공개, 장면 에셋, 멤버는 자기 조직 읽기만), `trailer-catalog`(비공개, 정책 없음 = 렌더러만), `trailer-previews`(공개, 미리보기·30초 미리듣기만). 원고·표지·사용자 음악은 코어 `org-files`, 렌더 결과는 코어 `exports`입니다.

적용 후 확인 쿼리

확인	쿼리	기대값
라우팅 기본값	`select count(*) from public.ai_routing_settings where org_id is null and app_id = 'trailer'`	16
강제 오픈소스	`select feature from public.ai_routing_settings where app_id = 'trailer' and force_open_source`	`dify.aux` 1행
플랜	`select code, signup_credits, monthly_credits, price_krw from public.plans where app_id = 'trailer' order by sort_order`	free 60/0/0, creator 0/800/29000, publisher 0/3000/99000
크레딧팩	`select code, credits, price_krw from public.credit_packs where app_id = 'trailer'`	pack_300 11,900, pack_1000 35,000, pack_3000 95,000
카탈로그	`select count(*) from public.grammars where status = 'published'`	6(seed 상태 확인. 다르면 05장 2.2절 기준)
pgvector	`select extname from pg_extension where extname = 'vector'`	1행

7.2 SQL 체커(PGlite)로 다시 확인

기획 단계 검증 결과(2026-10-15, PGlite): 모듈러 core 0000~0009 + seed + 제품 조각 16개 정순·역순 OK, 모놀리스 0001 + 0002 + seed OK, RLS 스모크 124/124(모듈러·모놀리스).

```
cd D:/WorkSpace/platform/docs/products/_tools-sqlcheck
npm ci
node check.mjs --vector mono \
  D:/WorkSpace/DEV/book-trailer/supabase/migrations/20261013000001_core_lite.sql \
  D:/WorkSpace/DEV/book-trailer/supabase/migrations/20261013000002_trailer_domain.sql \
  D:/WorkSpace/DEV/book-trailer/supabase/seed.sql
```

- 파일마다 `[mono] OK <파일>`이면 통과입니다. AI 테이블(pgvector) 때문에 `--vector`가 필수입니다.
- 체커는 적용 가능 여부만 봅니다. RLS 판정은 7.3절 스모크가 맡습니다.

7.3 RLS 스모크(`supabase/tests/rls_smoke.sql`)

124개 검사가 한 파일에 있습니다. 스키마 자리표시자 `@S@`(제품, 모놀리스 `public` / 모듈러 `trailer`), `@C@`(코어, `public` / `core`)를 씁니다.

머리 표시	의미
`-- SETUP` ~ `-- END SETUP`	관리 세션에서 사용자·조직·책·트레일러 등을 만듭니다
`-- AS service` / `anon` / `A` / `B` / `C` / `M` / `V`	이후 문장의 역할. A·B는 서로 다른 조직 owner, C는 엔타이틀먼트 없음, M·V는 A 조직의 member·viewer
`-- RUN`	다음 한 문장을 기대값 없이 실행
`-- TEST <이름> | expect=...`	`rows:N`, `ok`, `sqlstate:CODE`

가장 빠른 확인은 platform 저장소의 러너입니다(Supabase 없이 PGlite).

```
cd D:/WorkSpace/platform/docs
node products/book-trailer/supabase/monolith/tests/run-smoke.mjs --target mono   # 124/124
node products/book-trailer/supabase/monolith/tests/run-smoke.mjs --target mod    # 124/124
```

개발 폴더의 SQL을 고친 뒤 실제 Supabase로 확인하려면 TR-06-002 러너(`scripts/test-rls/`)를 쓰고, 그 전에는 `sed -e 's/@S@/public/g' -e 's/@C@/public/g'`로 치환본을 만들어 psql에서 문장 단위로 실행합니다(역할 전환 방법은 `develop/nomad-docs/nomad-docs_develop.md` 7.3절과 같음). 파일 전체를 한 번에 넣으면 역할 전환과 기대 오류 판정이 되지 않습니다.

7.4 DB 변경 규칙

- 정본은 모듈러 `2600_trailer.sql`이고 모놀리스 `0002`는 변환 스크립트로만 다시 만듭니다(TR-06-001, `CLAUDE.md` 2절 "DB 변경").
- 앱 저장소 사본은 `supabase/modular/2600_trailer.sql`에 두는 것을 제안합니다(`migrations/`에 두면 `db reset`이 적용하다 실패).
- 원격 `supabase db push`는 에이전트가 실행하지 않고 사람이 확인 후 실행합니다.

8. 6단계: 착수 전 결정 확정

`docs/file/02-요구사항/01_PRD.md` 8장의 결정 17개(D-14~D-17은 8.1절 인증 단계)를 확정합니다. 가정과 다르게 정하면 해당 산출물을 고치는 task를 따로 만듭니다.

ID	결정	권고	확정 시점
D-1	브랜드명	베타는 가칭, FLORA와 묶을지 출시 전 결정	랜딩(TR-06-044) 전
D-2	렌더 엔진	Remotion. 직원 4명 시점에 Company License	착수 전(TR-06-006 전)
D-3	FLORA 연동 시점	P1, 모듈러 배포에서만	W7 전
D-4	가격·단가	베타 4주 원가 측정 후. 지금 값은 상대 비율	베타 4주 후. 단가 상수(TR-06-013)는 제안값으로 진행
D-5	인용 한도	80/300자 시작	착수 전(`runtime_config` seed 값)
D-6	타인 도서 트레일러	Phase 3	확인만
D-7	음악 카탈로그	커미션 + 상업 라이선스 혼합 30곡	W3 전(TR-06-029·045). 곡이 없으면 무음·사용자 업로드로 진행
D-8	무료 플랜 범위	30초 1편(가입 60크레딧)	착수 전
D-9	작가 목소리 클론	Phase 2	확인만
D-10	다채널 업로드	P1 YouTube만, 다채널은 nomad-video	W7 전
D-11	도서정가제 규칙	법률 확인 전 FAIL	규칙 사전(TR-06-023) 전
D-12	화이트라벨	출판사 계약 3곳 이상이면	Phase 2 계획 시
D-13	YouTube 합성 콘텐츠 표시	API 미확인, 수동 안내	TR-06-058 전 공식 문서 확인
D-14	카카오 이메일 정책	선택 + 보충 입력	G2 판정 전
D-15	네이버 경로	스파이크로 결정	C2 첫 단계
D-16	이메일 가입 유지	유지	G1 판정 전
D-17	Google 클라이언트 분리	로그인용·YouTube 채널용 분리	G1 판정 전

산출물 승인 절차: 결정을 반영하는 task → 반영 후 `docs/file/_index.md` 상태를 "초안"에서 "승인"으로, 변경 이력 기록 → 개발 task는 "승인" 산출물만 baseline으로 씁니다.

9. 7단계: 작업을 task로 변환

9.1 작업과 웨이브 (정본: `docs/file/01-관리/06_AGENT_PLAN.md` 2~3절)

작업은 TR-06-001~060, 모두 60개입니다. TR-06-010은 사람이 하는 디자인 비주얼 검토입니다. P1은 054~060입니다.

웨이브	작업 ID	진입 조건	산출물
W0	001, 003, 007 → 002, 004, 005, 006, 015	없음	로컬 DB, web·오케스트레이터·렌더러·AI 서비스 골격, Mock 로그인
W1	012, 013, 016, 018, 019, 026, 049	W0	계약 스키마, 결제, AI 게이트, 업로드, 책·장면 API, CI
W1-D	008 → 009 → 010(사람) → 011	007	디자인 시스템 2~4단계. 011이 모든 화면 작업의 진입 조건
W2	017, 020, 021, 022, 024, 029, 030, 050	W1	AI 보안 테스트, 리다이렉트, 표지 팔레트, 분석 그래프, 실행 서버, 음악, 컴포지션, purge
W3	023, 027, 028, 031, 032, 045	W2	브리프, 이미지·내레이션, 렌더, 미리보기, 운영 콘솔
W4	025, 033~036, 038~040, 042~044, 051, 052 → 037, 041	W3 + 011	스토리보드·메타, 전체 화면, 코치·음성·위키, 랜딩, 법적 고지, 크론

웨이브	작업 ID	진입 조건	산출물
W5	046, 047, 048 → 014	W4	결제 E2E, 통합·렌더 회귀·E2E → 공개 베타 후보
W6	053	G1	Google + 이메일 로그인 → 공개 베타
W7	054~060	G2 / 베타 피드백	카카오·네이버, FLORA, 승인 링크, YouTube, BPM 컷, 팀 초대

첫 검증을 더 빠르게(축소 릴리스 v0, 제안): 9:16 30초 한 포맷, 문법 2종, 내레이션 오픈소스만, 브리프 → 스토리보드 → 렌더 직행으로 작업 약 22개(001~007, 012, 013, 015, 016, 018, 019, 022~026, 028, 030, 031, 033~036 일부)를 먼저 묶어 대행 납품(1.1절)에 내부 도구로 씁니다. 화면은 최소, 랜딩·코치·위키·음성은 뒤로 미룹니다.

9.2 모델과 에이전트 매핑 (주 Opus, 부 Sonnet, 고문 Fable)

06장 권장 모델	작업 성격	베이스에서 맡는 쪽
Fable	DB·RLS(001, 002), 결제(013), AI 게이트·보안(016, 017), 렌더 파이프라인의 워터마크·AI 표시 결정(031), 인증 B·C2(053, 055), FLORA·승인 링크(056, 057)	오케스트레이터(Opus)가 설계, `plan-challenger` (고문)가 착수 전 검토, 구현은 도메인 expert(Sonnet)
Opus	골격, 그래프 4종, 파서, 계약 스키마, 이미지·TTS 단계, 컴포지션, 테스트 설계, 운영 콘솔	`claude-main` 설계 → `nextjs-expert` · `fastapi-expert` (Sonnet) 구현
Sonnet	디자인 시스템 구현, 화면, 미리보기, 랜딩·3D	`nextjs-expert`
Haiku	CI(049), 법적 고지(051)	오케스트레이터 직접
사람	디자인 비주얼 검토 판정(010, 준비는 Sonnet), G1·G2 판정	대표님

작업 수 비율(60개, `06_AGENT_PLAN.md` 4절): Fable 10(16.7%), Opus 34(56.7%), Sonnet 14(23.3%, TR-06-010 판정은 사람), Haiku 2(3.3%). 규모 가중(합계 119)은 Fable 17.6%, Opus 56.3%, Sonnet 24.4%, Haiku 1.7%입니다.

9.3 task 폴더 만들기

```
tasks/TR-06-001-db-apply/
├── task.md      baseline에 산출물 ID, workers_approved, write_scope
├── context.md
└── log.md       [APPROVAL], [DECISION] 기록
```

```
baseline:
  - id: DOC-DES-004
    path: docs/file/03-설계/05_DATA_MODEL.md
  - id: DOC-CORE-004
    path: docs/file/03-설계/공용코어/DB_CONVENTIONS.md
  - id: DOC-CORE-003
    path: docs/file/03-설계/공용코어/SECURITY_BASELINE.md
  - id: DOC-MGT-002
    path: docs/file/01-관리/06_AGENT_PLAN.md   # TR-06-001 수용 기준
```

`write_scope`는 06장 2절 "소유 파일/디렉토리" 열을 5.2절 경로 표로 바꿔 적습니다. 한 경로는 한 작업만 씁니다.

9.4 요청 프롬프트 예시 (W0 첫 작업)

```
D:/WorkSpace/DEV/book-trailer/docs/file 의 모든 파일(프로젝트 산출물)을 최대한 자세히 읽고 분석한다.
(이미 읽은 파일은 skip. docs/file/_index.md 가 없으면 deliverable-first 스킬 절차로 산출물부터 추출한다.)
```

```
[요구사항]
TR-06-001: supabase/migrations 의 core_lite 와 trailer_domain, seed 를 로컬 Supabase 에 적용하고
타입을 생성한다(단독 프로젝트, public 스키마). 완료 조건 = ① supabase db reset 오류 0건
② ai_routing_settings 의 app_id='trailer' 행 16건, force_open_source 1건(dify.aux)
③ 플랜 free/creator/publisher, 크레딧팩 3종, 문법 6종·템플릿 6종 확인
④ supabase/tests/rls_smoke.sql 124항목 통과(platform 러너 또는 수동)
⑤ supabase gen types typescript --local --schema public 결과를 apps/web/src/types/supabase.ts 에 저장
⑥ PGlite 체커(check.mjs --vector mono 0001 0002 seed) OK
⑦ 06_AGENT_PLAN.md 의 TR-06-001 수용 기준을 단독(모놀리스) 기준으로 충족.

[작업의 특성]
DB만 진행. 원격 db push 금지. 작업 대상 경로: supabase/(tests/ 제외), scripts/db/, apps/web/src/types/

이 요구와 근거 산출물(baseline)로 작업을 만들어 진행한다.
승인·결정 사항은 묻지 말고 먼저 진행한 뒤, 임의로 결정한 내용은 log.md 에 [DECISION] 으로 남기고 종료 시 [DECISION-SUMMARY] 로 정리한다.

main worker 는 claude-main 이 진행한다.
main worker 의 서브 에이전트는 supabase-expert 가 진행한다.
review role worker 는 claude-reviewer 가 진행한다.
review role worker 의 서브 에이전트는 plan-challenger, review-expert 가 병렬 진행한다.
```

10. 8단계: 웨이브별 개발 사이클

순서	내용	도구
1	웨이브 시작 전 결정(8절) 확정, 산출물 "승인" 확인	`_index.md`
2	task 생성, baseline, `workers_approved` · `write_scope`	`_templates/task.md`
3	착수 전 계획 검토	`plan-challenger`
4	구현	도메인 expert, `gate.sh` · `scope_check.sh`
5	리뷰: 요구사항 → 설계 → 구현 → 테스트 추적성	`claude-reviewer` , `review-expert`
6	개발 위키 갱신(12절)	`llm-wiki` 스킬
7	완료 처리, 웨이브 종료 시 다음 웨이브	-

- **공개 베타 조건**: W5까지 녹색 + W6(Stage B) + `06_AGENT_PLAN.md` 7절 완료 정의. P1(054~060) 제외.
- 화면 작업은 디자인 시스템 4단계(TR-06-011) 뒤에 시작합니다.
- 구현이 산출물과 달라지면 코드를 조용히 맞추지 말고 산출물 갱신 task를 만듭니다.

11. AI 계층 구축 (book-trailer)

정본: `docs/file/03-설계/공용코어/AI_ARCHITECTURE.md` , `VOICE_ARCHITECTURE.md` , `LLM_WIKI.md` , 제품 `02_ARCHITECTURE.md` 5절, `04_API.md` 2.1·4·6절

구성	내용	작업
FastAPI AI 서비스	공용 ai-service + trailer 확장(`app/ai/graphs/trailer_*.py` , `app/ai/products/trailer/` , `app/ai/tools/trailer/` , `app/ai/compliance/trailer.py`). JWKS 검증 `aud=platform-ai:trailer`	015
라우팅	seed 16행. 분석·프롬프트 영문화·compliance·위키는 오픈소스 우선, 브리프·스토리보드·메타·채팅은 상용 우선, `dify.aux` 오픈소스 강제, 임베딩 `oss-embed` 고정, 내레이션 TTS는 ElevenLabs 우선(`02_ARCHITECTURE.md` 5.1절). 코드에 제공자 하드코딩 금지	015, 017
국외 이전 동의	온보딩에서 `set_ai_cross_border_ack` . 미동의 조직은 오픈소스만, 내레이션도 오픈소스 단가	016

구성	내용	작업
원고 파서·RAG	5형식 파싱, 스캔 PDF 오류, 책별 컬렉션 `trailer.knowledge`, 원문은 Storage에만	018, 022
그래프	`trailer.digest`(장 나누기 → 요약 → 인물·주제 → 색인 → 위키 초안), `trailer.brief`(훅 후보 → 인용 후보 → 원고 대조 → 스포일러 필터 → compliance → 3안), `trailer.storyboard`(문법 비트 → 장면 → 길이·노출 시간 검증), `trailer.meta`. 결과 저장은 ai-service가 `finish_run` 1회	022, 023, 025, 039
이미지	오케스트레이터가 fal 호출. 프롬프트 영문화(`trailer.prompt_en`, 라우팅) + 스타일 프리셋 + 팔레트 + 금지 접미어 + 고정 seed	027
내레이션	같은 목소리 규칙(첫 성공 시 고정, 같은 엔진 재시도 3회, 대체 없음)	028
코치 채팅	Dify `trailer-chat`, 입력 `platform_conversation_id`만, `/tools/answer`가 대화에서 조직·책을 찾고 출처(장·문단) 필수	040, 041
음성	음성 대화·받아쓰기·읽어 주기, 60초 상한, 녹음 미저장	042
보안 회귀	토큰 위조·만료, 대화 매핑, 국외 동의 필터, 다른 책 컬렉션 검색 0	017

순서: 공용 인프라(Dify, 오픈소스 LLM·임베딩·음성 서버, 없으면 OpenAI 호환 모의 서버) → ai-service 설치 → 그래프 → 오케스트레이터 단계. `OSS_VOICE_URL`은 대체 경로가 없어 없으면 기동 실패입니다(`02_ARCHITECTURE.md` 8절).

12. 렌더 파이프라인 (이 제품 고유)

정본: `02_ARCHITECTURE.md` 4절, `04_API.md` 4.4절, `08_DESIGN_SYSTEM.md` 5~10장

단계	내용	작업
스냅샷	`start_render`가 장면·에셋·키트·문법·템플릿·음악을 `renders.snapshot`으로 고정, 워터마크·AI 표시 결정, 선차감, enqueue(`trailer.render`)	031
props	렌더러가 스냅샷 → 컴포지션 props(zod 검증). Storage 경로 밖 fetch 금지	031
컴포지션	포맷 5종 x 장면 컴포넌트 8종, 안전 영역, `u` 그리드, 타입 스케일, 모션 프리셋, 자막, 믹스(덕킹 -12dB)	012, 030
출력	MP4(H.264/AAC), 포스터 PNG(제목 장면), SRT, 플랫폼 ZIP → `exports` 버킷 → `finish_render`	031
미리보기	브라우저 Remotion Player(프록시 에셋, 워터마크 오버레이, 다운로드 없음), 서버 480p(항상 워터마크)	032
품질 회귀	같은 스냅샷 프레임 해시 일치, 장면 8 x 포맷 4 스냅샷, 안전 영역 침범 0, 한글 서체, 60초 1080p p95 6분	047

- 렌더러 컨테이너 기동 확인(TR-06-006): `docker build services/trailer-renderer` → 1초 테스트 컴포지션 렌더 → 네트워크를 끈 상태에서 서체 8종 로드.
- 동시 렌더는 조직당 2(`runtime_config`), 렌더러 동시성 `RENDER_CONCURRENCY` 기본 2. Railway CPU 인스턴스 기준 벤치마크 결과를 `docs/wiki/decisions/`에 남깁니다.

13. 메인 디자인과 디자인 시스템

이 제품의 디자인 시스템은 두 층입니다(`docs/file/03-설계/08_DESIGN_SYSTEM.md` 0장).

층	무엇	정답 소스(코드)	작업
A 앱 UI	작업 화면(교정지·백지·인쇄 먹·연필심·형광 밑줄·상영관 팔레트, 나눔명조 ExtraBold + IBM Plex Sans KR)	`apps/web/src/styles/tokens.css`, `components/ui/`	007~011
B 영상	트레일러 안(책별 브랜드 키트, OFL 서체 8종, 캔버스·안전 영역·타입 스케일, 문법 6종, 모션, 자막, 이미지 스타일, 장면 컴포넌트)	`packages/trailer-compositions/src/tokens`, `scenes/`	012, 021, 030

두 층을 섞지 않습니다: 앱 화면에 책 키트 색을 쓰지 않고, 영상에 앱 팔레트를 쓰지 않습니다(작업실 미리보기 안만 층 B).

13.1 디자인 시스템 4단계 (층 A, `DESIGN_SYSTEM.md` 4장)

단계	작업	담당
1. 레퍼런스 확정	작업실 Atlassian + 패턴 J, 목록·모니터 Primer, AI 채팅 Astryx 패턴, 랜딩 `HERO_3D.md`. 라이브러리는 설치하지 않음	TR-06-007, 고문 검토
2. 가진 것 위에서	`08_DESIGN_SYSTEM.md` 3.7절 값 → `tokens.css` (Tailwind CSS v4 `@theme`), 기본 컴포넌트와 `/dev/components`	TR-06-008, 009
3. 비주얼은 사람이	라이트/다크 x 컴팩트/기본 스크린샷으로 수치 수정. `--color-highlight`(형광 밑줄)가 선택·인용 강조에만 쓰였는지 확인	TR-06-010(사람)
4. 정답 한 곳	코드를 정답으로 `DESIGN.md`, `COMPONENTS.md` 갱신	TR-06-011

`docs/file/03-설계/디자인/DESIGN.md`, `COMPONENTS.md`는 `08_DESIGN_SYSTEM.md` 3.7절·11.3절 초기값으로 채워져 들어옵니다. 제품 컴포넌트(SceneStrip, ScenePropertyPanel, BriefCompare, QuoteHighlight, RightsGate, PaletteEditor, RenderMonitorRow, FormatPicker, SafeZoneOverlay 등)는 기본 컴포넌트 뒤에 1개씩 맡깁니다(`08_DESIGN_SYSTEM.md` 11.2절, 13장 작업 지시 예시).

13.2 영상 디자인(층 B)

순서	작업	확인
1	영상 토큰 상수(포맷별 `u`, 타입 스케일, 안전 영역, `minReadMs`)	단위 테스트(TR-06-012)
2	표지 팔레트 추출(CIELAB k-means k=8 seed 7, 대비 보정)	고정 표지 10장 기대값(TR-06-021)
3	장면 컴포넌트 8종 x 포맷 5종	스냅샷 + 안전 영역 침범 0(TR-06-030, 047)
4	이미지 스타일 프리셋·금지 접미어	일관성 검수 기준표(`08_DESIGN_SYSTEM.md` 8.3절, 사람)
5	변주 규칙(같은 템플릿 반복 줄이기)	`08_DESIGN_SYSTEM.md` 10장

13.3 메인 히어로 (정본: `HERO_3D.md`, `03_SCREENS_UX.md` 4.5절)

소재: 펼친 책의 띠지(형광 노랑 종이 띠)가 풀리며 세로 9:16 프레임으로 접히고, 그 안에서 장면 카드 3장(인용·이미지·제목)이 차례로 넘어간 뒤 재생 표시. 같은 책의 16:9·1:1 프레임이 뒤에 겹쳐 놓입니다. 의미는 "책이 규격별 트레일러가 된다"입니다.

기기	구성
PC·태블릿	실시간 3D(장면 카드 텍스처는 실제 샘플 트레일러 프레임)
모바일	띠지 → 세로 프레임 1회 접힘만, 카드 1장
3D 영상 단계	같은 코드로 오프라인 렌더한 8~12초 무음 루프(2MB, 모바일 1MB), 포스터 = 정지 렌더, 킬 스위치 `NEXT_PUBLIC_HERO_VIDEO_ENABLED`
정지 이미지	reduced-motion, 데이터 절약, 영상 실패

이 제품은 3D 영상 루프를 Remotion 렌더러로도 만들 수 있습니다(제안: 3D 장면을 `@remotion/three`로 감싸 오프라인 렌더하면 별도 녹화 스크립트가 필요 없음, 호환성 구현 시 확인). 재질 색은 토큰만, 3D 안에 글자·숫자 없음.

13.4 시각 레퍼런스 (정본: `03_SCREENS_UX.md` 4.1절, `REFERENCES.md`)

구분	제목	URL	가져올 것	적용 화면
랜딩	Codapress Publishing	https://www.awwwards.com/sites/codapress-publishing	표지를 고르면 그 책의 결과물이 열리는 갤러리 동선	`/samples`, 샘플 섹션
랜딩	Physical Matter by Runway	https://www.awwwards.com/sites/physical-matter-by-runway	무음 루프 영상 + 포스터 처리	히어로 3D 영상 단계
랜딩	Join Radio	https://www.awwwards.com/sites/join-radio	선택에 따라 그래픽이 바뀌는 방식 → 문법 6종 전환 섹션	랜딩 "장르 문법"
앱	Video Editing Dashboard	https://dribbble.com/shots/25186663-Video-Editing-Dashboard	미리보기 + 하단 가로 스트립 위계 (트랙이 아니라 장면 칸 한 줄)	작업실
앱	AI book app	https://dribbble.com/shots/23707376-AI-book-app	AI 결과 여러 안 나란히 + 하나 고르기	브리프 3안 비교

- 같은 레퍼런스를 다른 제품도 씁니다. 03장 4.1절 "다른 제품의 쓰임" 열과 다르게 가져옵니다.
- 이 표는 `REFERENCES.md` book-trailer 절(2026-10-16 추가)과 같습니다. 이 제품 전용 후보(북트레일러·출판 마케팅 사이트)는 URL을 확인한 뒤 그 절에 먼저 추가합니다.
- 영감용이지 복제용이 아닙니다. Dribbble 링크는 작업 전에 브라우저에서 한 번 엽니다.

14. 원본 이식

정본: `docs/file/04-구현/07_SOURCE_MIGRATION.md`

- 전용 원본 저장소가 없습니다. 이식 원천(nomad-video, toon-factory, flora-publishing, nomad-docs 기획, 리뷰 2건)은 **읽기 전용**이고 아카이브 대상도 없습니다.
- 실제로 옮기는 것은 규칙 문장(같은 목소리 규칙), 라이선스 CHECK 구조(이미 `2600`에 반영), RFC 5987 한글 파일명뿐입니다. 나머지는 패턴만 보고 새로 작성합니다.
- 리뷰 "높음" 18건(NOMAD Shorts Maker 12, Toon-Factory 6)의 차단 설계는 비보안 8건 `02_ARCHITECTURE.md` 6절, 보안·데이터 10건 `05_DATA_MODEL.md` 7절입니다. 해당 task 리뷰 단계에서 차단 여부를 확인합니다.
- 폐기 TS-01~12(BullMQ·Python 미디어 워커, FFmpeg 단일 엔진, Fabric.js 캔버스, LoRA, 브라우저 렌더, 메모리 버퍼 업로드, localStorage Mock 등)는 옮기지 않습니다.

15. 공용 모듈 경계

항목	규칙
코어 재정의 금지	auth, org, credits, toss, webhooks, oauth, storage, audit, jobs, compliance, ai-gateway, ai-service, dify, core-voice, llm-wiki, sns-uploader는 import·호출만
키 위치	LLM 키는 `apps/api`에만. fal·ElevenLabs는 오케스트레이터(ElevenLabs는 web 서버 음성 대화도). 서명 개인키는 web 서버만. 렌더러에는 AI 키 없음
sns-uploader	YouTube 업로드(P1)는 nomad-video 소유 공용 모듈의 소비자
렌더러	지금은 이 제품 소유. toon-factory MP4·nomad-video와 공유할지 R-T8에서 판단
공용 요청	R-T1(모듈러 seed), R-T2(번호·SQLSTATE), R-T4(잡 토큰 `parse` scope), R-T5(인증 매트릭스), R-T6(sns-uploader 소비자), R-T7(REFERENCES 절)은 2026-10-16 반영. 남은 것은 R-T3 서점 제휴 범용화, R-T8 렌더러 공용화(둘 다 Phase 2, `06_AGENT_PLAN.md` 6절)

16. 검증과 완료 정의

조건	확인
빌드·타입·린트·단위	`pnpm -F web lint && pnpm -F web typecheck && pnpm -F web test && pnpm -F web build`, 오케스트레이터·렌더러·컴포지션 패키지 test
Python	`pytest apps/api -k trailer`, `ruff check apps/api`
SQL	`check.mjs --vector mono 0001 0002 seed` OK, 모듈러는 platform에서 `node check.mjs --vector mod <0000~0009> seed.sql 2600_trailer.sql` OK, 모놀리스 변환 `--check` 불일치 0
RLS 스모크	124항목(모놀리스 필수, 모듈러 정본 확인)
결제 E2E	`npx playwright test e2e/billing` (Toss 테스트 키)
흐름	통합(TR-06-046, 가짜 제공자 + 실제 DB·렌더러), E2E(TR-06-048, 무료 체험 흐름)
렌더	TR-06-047 결정성·안전 영역·서체, 60초 1080p p95 6분 이하 기록
AI	평가셋: 장 나누기, 인용 대조 100%, 금지 문구(실적·보장·할인 15% 초과) FAIL, 코치 재현율 0.8(제안)
법·문구	01장 6장 반영, 랜딩·영상 카피 금지 문구 0건, 엔드카드 AI 표시
디자인	DESIGN_PRINCIPLES 7단계 체크리스트, `tailwind.config.js`·hex 하드코딩 0
인증	Stage A 수용 기준(TR-06-004), Stage B `AUTH_ROADMAP.md` 5.3절 B-1~B-6(19장)

17. 위험과 주의

위험	대응
저작권(원고·인용)	권리 확인 없으면 분석·렌더 불가(`TR003`), 인용은 원고 대조 통과분만·80/300자. 타인 도서 트레일러는 Phase 3(D-6)
표시광고·도서정가제	금지 문구 FAIL, 15% 초과 할인 표현 FAIL(D-11, 법률 확인 미완). 판매·순위 문구는 근거 입력 필수
AI 생성물 표시	엔드카드 AI 표시를 DB CHECK로 강제. 플랫폼 합성 콘텐츠 표시 API는 미확인(D-13)
플랫폼 정책(양산형)	같은 템플릿 7일 3편 경고, 변주 규칙. 대량 자동 업로드 기능을 만들지 않습니다
음악 라이선스	검증 기록 없는 곡은 공개 불가. 카탈로그 확보(D-7) 전에는 무음·사용자 업로드
렌더 비용·시간	헤드리스 Chromium 렌더는 CPU를 많이 씁니다. 벤치마크(TR-06-047) 후 렌더 단가(D-4)와 동시성 조정
이미지 원가	세트 원가의 절반 이상. 기본 모델 선택과 사용자 이미지 업로드 유도(가격 플랜 3장)
Remotion 라이선스	직원 4명 시점 Company License(D-2). 인원 산정 기준 확인
같은 목소리	제공자 장애 시 다른 엔진으로 대체하지 않아 실패가 늘 수 있습니다. 실패는 환불, 사용자에게 "다시 녹음" 안내
작업 수	60개. 9.1절 축소 릴리스 v0와 1.1절 대행 검증을 먼저 고려
비밀키	`.env*` 커밋 금지, 키 값을 로그·위키에 쓰지 않음

18. 체크리스트

- [ ] 1.1절 린 검증 시작(대행 상품 등록, 수동 제작 1건)
- [ ] 2절 도구 설치 확인, Remotion 라이선스 조건 확인
- [ ] setup 스크립트 dry-run → 실행, 무결성 점검 PASS
- [ ] `_shared/project-profile.md` 작성(6절), `claude` 실행 후 `/agent-check`

- [ ] 공용 인프라: Dify, 오픈소스 LLM·임베딩·음성 서버(또는 모의 서버)
- [ ] `supabase db reset` 성공, 라우팅 16행·강제 오픈소스 1행·플랜·카탈로그 확인(7.1절)
- [ ] PGlite 체커 OK, RLS 스모크 124항목(7.2~7.3절)
- [ ] 결정 D-1~D-17 확정, 산출물 "승인" 전환(8절)
- [ ] W0 task 생성과 진행(9절), 렌더러 컨테이너 1초 렌더 확인(12절)
- [ ] 디자인 시스템 1~4단계(13.1), 영상 토큰·팔레트 추출(13.2)
- [ ] Stage A: 시드 페르소나 로컬 로그인(19.2절)
- [ ] W1~W5 → G1 판정, `[DECISION] G1 통과`(19.3절)
- [ ] W6 Stage B → 공개 베타
- [ ] 베타 4주 원가 측정 → D-4 가격 확정, G2 판정 → W7

19. 인증 단계: Mock → Google → 카카오·네이버

정본: `docs/file/03-설계/공용코어/AUTH_ROADMAP.md`(확정 결정 22). 제품 반영: `01_PRD.md` TR-F-001·8.1절, `02_ARCHITECTURE.md` 9절, `03_SCREENS_UX.md` 7절, `04_API.md` 8절, `05_DATA_MODEL.md` 10절, `06_AGENT_PLAN.md` 8절, `07_SOURCE_MIGRATION.md` 6절, `CLAUDE.md` "인증 단계 규칙". 외부 콘솔 화면별 절차는 공용 지시서 `D:/WorkSpace/platform/docs/develop/_shared/auth/README.md`를 따릅니다.

19.1 단계 요약

단계	언제	켜지는 로그인	작업	끝나는 조건
A Mock	W0~W5	로컬 시드 페르소나 원클릭 로그인, 배지 `MOCK AUTH · 로컬 전용`	TR-06-004(P0, Opus)	G1
B Google	G1 후(W6)	Google + 이메일(가입·메일 인증·재설정), 첫 OAuth 로그인 뒤 약관 동의	TR-06-053(P0, Fable)	G2
C1 카카오	G2 후(W7)	카카오	TR-06-054(P1, Opus)	`AUTH_ROADMAP.md` 7.1.3절
C2 네이버	G2 후(W7)	네이버(`custom:naver` 또는 브리지)	TR-06-055(P1, Fable)	`AUTH_ROADMAP.md` 7.2.4절

19.2 Stage A 로컬 실행 순서

순서	명령·행동	설명
1	`supabase start` → `supabase db reset`	7.1절
2	`apps/web/.env.local`에 `AUTH_MODE=mock`, `DEV_SEED_PASSWORD=<긴 임의 값>`	커밋 금지, `NEXT_PUBLIC_*` 금지
3	`node scripts/dev/seed-users.mjs --app trailer`	페르소나 `owner`, `creator_owner`, `pub_owner`, `pub_member`, `pub_viewer`, `outsider`, `no_app`, `suspended`, `ops`(`05_DATA_MODEL.md` 10절). 원격 URL이면 즉시 종료
4	`pnpm -F web dev` 후 `/login`	개발용 패널에서 페르소나 버튼. `suspended`는 거부, `no_app`은 온보딩
5	E2E	Playwright `globalSetup`이 페르소나별 `storageState` 저장(TR-06-048)

확인 예: `owner`는 잔액 60(가입 크레딧)이고 30초 9:16 체험이 워터마크와 함께 렌더됩니다. 1080p를 고르면 `TR005`. `pub_viewer`는 읽기만, `outsider`는 다른 조직 책 0행.

19.3 G1 체크리스트 (사람이 판정)

- [ ] P0 기능 작업 완료(인증 B·C 제외)
- [ ] 통합·E2E(TR-06-046, 048) Mock 페르소나로 녹색
- [ ] RLS 스모크 124항목, 결제 E2E(TR-06-014) 녹색
- [ ] CI(TR-06-049): 인증 모듈 밖 `AUTH_MODE` 참조 0, `mock-auth`·`nomad_role`·`nomad_plan` 0
- [ ] 외부 준비: 로그인 전용 Google Cloud 프로젝트(YouTube 채널 연결용과 분리, D-17), OAuth 동의 화면, 개인정보처리방침·이용약관 URL
- [ ] `[DECISION] G1 통과` 기록

19.4 Stage B·C 외부 설정

`develop/nomad-docs/nomad-docs_develop.md` 19.4절(Google), 19.6절(카카오), 19.7절(네이버 스파이크)과 같은 순서입니다. 다른 점:

- 배포 env `AUTH_PROVIDERS=google,email`(C 단계에서 `,kakao`, `,naver`), 약관 버전 변수 `TRAILER_TERMS_VERSION`(제안).
- 로그인용 Google 클라이언트와 YouTube 채널 연결용 Google 클라이언트를 같은 프로젝트에 두지 않습니다(D-17). 채널 연결 OAuth는 `core.oauth_states`로, 로그인은 Supabase Auth 제공자로만.
- 통합 DB 제품끼리 Google·카카오·네이버 앱을 1세트 공유하면(공용 지시서 AUTH-D1) Redirect URLs에 이 제품 도메인만 더합니다.

19.5 이 제품의 인증 작업 ID

단계	작업	우선순위·규모·모델	웨이브
A	TR-06-004	P0, M, Opus	W0
B	TR-06-053	P0, M, Fable	W6(G1 후)
C1	TR-06-054	P1, S, Opus	W7(G2 후)
C2	TR-06-055	P1, M, Fable	W7(G2 후)

부록 A. 스크립트 없이 손으로 할 때

동작	Git Bash / Linux	Windows PowerShell
1 복사	`cp -r "D:/WorkSpace/DEV/MultiAgent-Service-Fullstack-Tree-Framwork" "D:/WorkSpace/DEV/book-trailer"`	`Copy-Item "D:\WorkSpace\DEV\MultiAgent-Service-Fullstack-Tree-Framwork" "D:\WorkSpace\DEV\book-trailer" -Recurse`
2 이동	`cd "D:/WorkSpace/DEV/book-trailer"`	`Set-Location "D:\WorkSpace\DEV\book-trailer"`
3 초기화	`bash init-project.sh book-trailer --reset-git`	`& "$env:ProgramFiles\Git\bin\bash.exe" init-project.sh book-trailer --reset-git`
4 붙여넣기	`cp -r "D:/WorkSpace/platform/docs/develop/book-trailer/book-trailer-root/." "D:/WorkSpace/DEV/book-trailer/"`	`Copy-Item "D:\WorkSpace\platform\docs\develop\book-trailer\book-trailer-root\*" "D:\WorkSpace\DEV\book-trailer" -Recurse -Force` (숨김 폴더 `.claude`도 같은 방법으로)
5 docs/etc 삭제	`rm -rf docs/etc`	`Remove-Item docs\etc -Recurse -Force`

동작	Git Bash / Linux	Windows PowerShell
6 모델 지정	`.claude/agents/plan-challenger.md` 의 `model: opus` → `model: claude-fable-5-1`, `nextjs-expert.md` · `fastapi-expert.md` · `supabase-expert.md` frontmatter에 `model: sonnet` 추가	같음(편집기)
7 점검	`bash _shared/check-invariants.sh`	`& "$env:ProgramFiles\Git\bin\bash.exe" _shared/check-invariants.sh`

묶음을 다시 만들 때(기획서가 바뀐 경우): `cd D:/ && PYTHONUTF8=1 python -I D:/WorkSpace/platform/docs/develop/_shared/tools/build_root.py book-trailer` → 마지막 줄 "묶음 안 누락 0, 절대경로 누락 0" 확인.